

INERLAB-R

Sistema modular en Python para análisis de inercia rotacional

Descripción del pipeline computacional

Haniel Chaves Felipe Chirivi Tatiana Valero

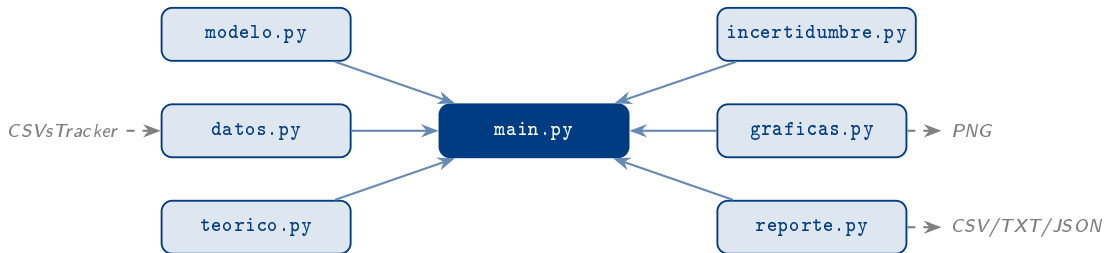
Universidad Industrial de Santander — Laboratorio de Física 1

Semestre I · 2026

Contenido

- 1 Arquitectura general
- 2 `modelo.py`
- 3 `datos.py`
- 4 `incertidumbre.py`
- 5 `teorico.py`
- 6 `graficas.py` y `reporte.py`
- 7 `main.py`
- 8 Comparación y conclusión

Arquitectura — 7 módulos independientes



- `main.py` solo orquesta — no contiene física
- Cada módulo importable de forma independiente
- Fallback automático si Tracker no exportó θ

- `numpy`, `pandas`, `matplotlib`
- `scipy.stats` para regresión y prueba t
- Sin dependencias propietarias ni licencias

Modelo físico

$$T = m_{\text{ef}}(g - a) \quad (\text{tensión})$$

$$\tau = r \cdot T \quad (\text{torque})$$

$$I_{\text{exp}} = \tau / \alpha \quad (\text{inercia exp.})$$

$$I_{\text{net}} = I_{\text{exp}} - I_{\text{ap}} \quad (\text{descuento})$$

$m_{\text{ef}} = m - m_{\text{fri}}$ descuenta la **masa de fricción** empírica.

```
1 def calcular_inercia_exp(  
2     m, g, r, a, alpha,  
3     m_friccion=0.0):  
4     # Corrección por masa de fricción  
5     m_ef = m - m_friccion  
6     T = m_ef * (g - a) # tensión  
7     tau = r * T # torque  
8     return tau / abs(alpha)  
9  
10 def descontar_inercia_aparato(  
11     I_total, I_aparato):  
12     # Resta inercia del riel/polea  
13     I_neta = I_total - I_aparato  
14     if I_neta < 0:  
15         raise ValueError("Revisar I_ap")  
16     return I_neta
```

Correcciones que otros códigos omiten

- (1) m_{fri} se descuenta *antes* de calcular T , no después.
- (2) I_{ap} se mide sin masa y se resta al total.

Dos métodos para α y a

Método 1 — Ajuste cuadrático de $\theta(t)$ ✓

Ajusta $\theta = \frac{1}{2}\alpha t^2 + \omega_0 t + \theta_0$ con covarianza del polinomio.

$\sigma_a = \sigma_\alpha \cdot r$ viene del ajuste, no es un valor fijo.

Método 2 — Regresión lineal de $\omega(t)$

Pendiente de $\omega = \alpha t + \omega_0$. Fallback automático si Tracker no exportó θ .

Detección automática de columnas

Reconoce t , time, tiempo; ω , omega, omega (rad/s); θ , angle (rad). Sin renombrar columnas a mano.

```
1 # Método 1: cuadrático en theta(t)
2 def calcular_a_desde_posicion(
3     t, theta, r_polea):
4     coefs, cov = np.polyfit(
5         t, theta, 2, cov=True)
6     c2 = coefs[0]
7     alpha_v = 2.0 * c2
8     s_alpha = 2.0 * np.sqrt(cov[0,0])
9     a = alpha_v * r_polea
10    s_a = s_alpha * r_polea
11    return {"a":a, "sigma_a":s_a,
12           "alpha":alpha_v, ...}
13
14 # Método 2: lineal en omega(t)
15 def calcular_alpha_regresion(t, w):
16     sl,_,_,_,se = \
17         stats.linregress(t, w)
18     return {"alpha":sl,
19           "sigma_alpha":se, ...}
```

Propagación analítica de σ_I

Para $I = r m_{\text{ef}}(g - a)/\alpha$:

$$\sigma_I = \sqrt{\left(\frac{\partial I}{\partial m} \sigma_m\right)^2 + \left(\frac{\partial I}{\partial r} \sigma_r\right)^2 + \left(\frac{\partial I}{\partial a} \sigma_a\right)^2 + \left(\frac{\partial I}{\partial \alpha} \sigma_\alpha\right)^2}$$

$$\frac{\partial I}{\partial m} = \frac{r(g-a)}{\alpha}, \quad \frac{\partial I}{\partial r} = \frac{m(g-a)}{\alpha}, \quad \frac{\partial I}{\partial a} = \frac{-rm}{\alpha}, \quad \frac{\partial I}{\partial \alpha} = \frac{-rm(g-a)}{\alpha^2}$$

Resultado adicional: $\sigma_{\text{rel}} = \sigma_I / I \times 100\%$

Prueba t -Student por radio

$H_0: \mu(I_{\text{exp}}) = I_{\text{teo}}$. $t = \frac{\bar{I}_{\text{exp}} - I_{\text{teo}}}{s/\sqrt{n}}$. Requiere $n \geq 3$; el sistema lanza advertencia si no.

```
1 def propagar_incetridumbre(  
2     m, sigma_m, r, sigma_r,  
3     g, a, sigma_a,  
4     alpha, sigma_alpha):  
5  
6     # Derivadas parciales analíticas  
7     dI_dm = r*(g-a)/abs(alpha)  
8     dI_dr = m*(g-a)/abs(alpha)  
9     dI_da = -r*m/abs(alpha)  
10    dI_dal = -r*m*(g-a)/alpha**2  
11  
12    sI = np.sqrt(  
13        (dI_dm *sigma_m      )**2 +  
14        (dI_dr *sigma_r      )**2 +  
15        (dI_da *sigma_a      )**2 +  
16        (dI_dal*sigma_alpha)**2 )  
17  
18    I_c = r*m*(g-a)/abs(alpha)  
19    return {  
20        "sigma_I":          sI,  
21        "sigma_rel_pct": sI/I_c*100,  
22        "contribuciones": {...} }
```

Experimento 1 — Masa puntual

$$I_{\text{teo}} = MR^2$$

Verifica la dependencia cuadrática para
 $R = 5, 10, 15, 20$ cm.

Experimento 2 — Teorema de Steiner

$$I_{\text{teo}} = M(R_1^2 + R_2^2)$$

Dos masas simétricas; se varía (R_1, R_2)
manteniendo $R_1 + R_2 = \text{cte}$.

```
1 # Experimento 1      masa puntual
2 def inercia_masa_puntual(M, R):
3     return M * R**2
4
5 # Arreglo para la curva teórica
6 def inercia_masa_puntual_array(
7     M, radios):
8     return M * np.asarray(radios)**2
9
10 # Experimento 2      dos masas iguales
11 def inercia_dos_masas(M, R1, R2):
12     return M * (R1**2 + R2**2)
13
14 # Steiner general
15 def steiner(I_cm, M, d):
16     return I_cm + M * d**2
17
18 # Error porcentual
19 def error_porcentual(I_exp, I_teo):
20     return abs(I_exp - I_teo) / I_teo * 100
```

Criterio de éxito del proyecto

4 gráficas automáticas

1. l_{exp} vs R^2 con recta teórica y M_{exp} en la leyenda
2. Paridad l_{exp} vs l_{teo} (línea identidad + barras de error)
3. Error % por radio (línea umbral 10 % en rojo)
4. $\omega(t)$ con regresión lineal (diagnóstico calidad de video)

3 formatos de exportación

CSV — tabla lista para el informe:

R [m]	l_{exp}	σ_l	Error %
0.05	...	...	2.1
0.10	...	...	0.6

TXT — reporte con pruebas t :

$|t|=2.14 < t_{\text{crit}}=4.30 \rightarrow H_0 \text{ ok}$

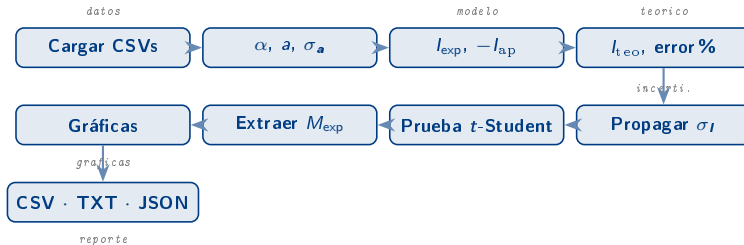
JSON — todos los resultados serializados para postproceso o comparación entre sesiones.

Nivel avanzado: M_{exp} desde pendiente

La pendiente del ajuste $l = M_{\text{exp}} R^2$ es una **medida independiente** de la masa. Se compara con la balanza:

$$\epsilon_M = |M_{\text{exp}} - M_{\text{bal}}| / M_{\text{bal}} \times 100$$

main.py — Orquestación del pipeline completo



```
1 # Sección de configuración
2 USE_DATOS_EJEMPLO = False # <- cambiar
3 G = 9.780 # m/s2 Bucaramanga
4 M_PUNTUAL = 0.200 # kg (balanza)
5 M_COLGANTE = 0.050 # kg
6 R_POLEA = 0.015 # m (calibrador)
7 M_FRICCION = 0.003 # kg (empirico)
8 I_APARATO = 2.5e-5 # kg*m2 (medido)
```

```
1 # Rutas a los CSVs de Tracker
2 RUTAS_EXP1 = [
3     ["datos/R5_r1.csv",
4      "datos/R5_r2.csv",
5      "datos/R5_r3.csv"], # R=5 cm
6     # ... (4 radios x 3 reps)
7 ]
8 # Ejecucion:
9 # python main.py
10 # Salida -> resultados/
```

Comparación: INERLAB-R vs. código de referencia

Aspecto	Código de referencia	INERLAB-R
Cálculo de a	<code>df["a"].mean()</code> fijo	Ajuste cuadrático $\theta(t)$
σ_a	Constante global 0.01	De covarianza del ajuste
Múltiples repeticiones	No	✓ $n \geq 3$ por radio
Corrección por fricción	No	✓ $m_{\text{ef}} = m - m_{\text{fri}}$
Descuento I_{ap}	No	✓ Medido y restado
Propagación σ_I	✓ Analítica	✓ $+$ σ_{rel}
Prueba t -Student	No	✓ Por radio / configuración
Extracción M_{exp}	No	✓ Pendiente de I vs R^2
Gráficas	2, sin recta teórica	4, con M_{exp} y umbral 10 %
Exportación	Solo CSV	CSV + TXT + JSON
Tamaño	≈ 120 líneas (1 script)	≈ 730 líneas (7 módulos)

Conclusión

El código de referencia es útil para verificar la lógica central. INERLAB-R cierra el **ciclo experimental completo**:

INERLAB-R

Pipeline computacional — Física 1

Haniel Chaves
2221147

Felipe Chirivi
2220658

Tatiana Valero
2200418

Universidad Industrial de Santander · Laboratorio de Mecánica · 2026

`python main.py`